



Wrocław
University
of Science
and Technology

Software Engineering

Database Design – Software Engineering Perspective

Marcin Jodłowiec

December, 2nd & December, 10th, 2025

Agenda

- Introduction
- Data Modeling Essentials
- Choosing Database Model
- Application Integration
- Design Principles & Anti-patterns

Introduction

Introduction

Why Databases Matter in Software Engineering

Why Databases Matter

In software engineering, the database is not just a storage mechanism:

- ▶ **Data outlives applications** – your data will survive multiple versions, rewrites, and technology changes
- ▶ **Quality of data model = quality of the system** – a poor data model leads to unmaintainable code
- ▶ **Performance depends on proper design** – most performance issues trace back to database design
- ▶ **Scalability constraints** – database choices limit or enable system growth
- ▶ **Maintenance costs** – bad design compounds technical debt exponentially

Database Design

The Software Engineer's Perspective

- ▶ Database design is not just for DBAs – every software engineer must understand it
- ▶ Your application architecture is constrained by your data model
- ▶ Good database design enables:
 - ▶ Clean application code
 - ▶ Efficient queries
 - ▶ Easy testing
 - ▶ Smooth deployments
 - ▶ System evolution
- ▶ Bad database design causes:
 - ▶ Complex, fragile code
 - ▶ Performance problems
 - ▶ Data integrity issues
 - ▶ Difficult migrations
 - ▶ Technical debt

Data Modeling Essentials

Data Modeling

From Domain to Database

The Journey

1. **Domain Model** – identify business entities and their relationships (from requirements analysis)
 2. **Logical Model** – structure data independently of technology (ER diagram)
 3. **Physical Model** – implement in specific database technology (SQL DDL, NoSQL schema)
- ▶ Start with your domain model (class diagrams from earlier lectures)
 - ▶ Translate domain entities to database entities
 - ▶ Preserve business rules and constraints
 - ▶ Consider data access patterns early

Entity-Relationship Modeling

The Basics

Core Concepts

- ▶ **Entities** – things that exist independently (Student, Course, Order)
- ▶ **Attributes** – properties of entities (name, email, price)
- ▶ **Relationships** – associations between entities (enrolls in, contains, orders)
- ▶ **Cardinalities** – how many of each entity participate
 - ▶ 1:1 (one-to-one) – Person has one Passport
 - ▶ 1:N (one-to-many) – Department has many Employees
 - ▶ M:N (many-to-many) – Students enroll in many Courses
- ▶ **Arity** – number of entities in a relationship (binary, ternary, etc.)

ER Modeling Symbols

Definitions:

entity something about which data is collected, stored, and maintained

attribute a characteristic of an entity

relationship an association between entities

entity type a class of entities that have the same set of attributes

record an ordered set of attribute values that describe an instance of an entity type

Symbols:

 entity type

 attribute

 relationship between entities

 one-to-one association

 one-to-many association

 many-to-many association

 partly optional association

 fully optional association

 mutually inclusive association

 mutually exclusive association

Examples:

One A is associated with one B:



One A is associated with one or more B's:



One or more A's are associated with one or more B's:



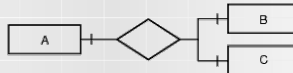
One A is associated with zero or one B:



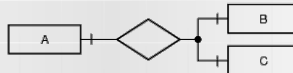
One A is associated with zero or more B's:



One A is associated with one B and one C:



One A is associated with one B or one C (but not both):

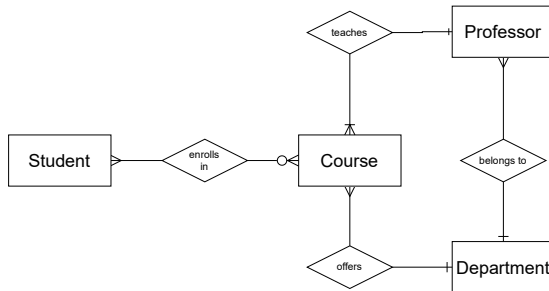


ER Modeling I

Practical Example: University System

Consider a university course management system:

- ▶ **Entities:** Student, Course, Professor, Department
- ▶ **Relationships:**
 - ▶ Student *enrolls in* Course (M:N)
 - ▶ Professor *teaches* Course (1:N)
 - ▶ Department *offers* Course (1:N)
 - ▶ Professor *belongs to* Department (N:1)



ER Modeling II

Practical Example: University System

Key observations:

- ▶ M:N relationships require an **associative entity** (Enrollment)
- ▶ Associative entities can have their own attributes (grade, enrollment_date)
- ▶ Every entity needs a **primary key** (unique identifier)
- ▶ Relationships become **foreign keys** in the physical model

Common implementation mistake (SQL level):

- ▶ Attempting to store M:N relationships using a single foreign key
- ▶ OR using comma-separated lists (violates 1NF)
 - ▶ This does not occur in conceptual ER modeling, but it *does* occur during physical implementation

Keys

Engineering Perspective

Primary Keys

Uniquely identify each row in a table

- ▶ **Natural keys** – business meaningful (email, SSN, ISBN)
- ▶ **Surrogate keys** – system generated (auto-increment ID, UUID)

Foreign Keys

Reference primary keys in other tables

- ▶ Enforce referential integrity
- ▶ Define cascade behavior (ON DELETE CASCADE, SET NULL, RESTRICT)

Engineering Choice: Surrogate keys are usually preferred

- ▶ Natural keys can change (email addresses, phone numbers)
- ▶ Surrogate keys are immutable and efficient
- ▶ Keep natural keys as UNIQUE constraints for business logic

Normalization

Why and When

The Goal

Eliminate data redundancy and update anomalies

- ▶ **1NF** – Atomic attributes (no repeating groups or arrays in cells)
- ▶ **2NF** – No partial dependencies (all attributes depend on entire key)
- ▶ **3NF** – No transitive dependencies (non-key attributes don't depend on other non-key attributes)

Practical Rule:

- ▶ Start normalized (usually 3NF is enough)
- ▶ Good domain model usually leads to normalized schema
- ▶ Denormalize only when you have proven performance problems
- ▶ Document every denormalization decision

Denormalization

When performance demands it

Valid reasons to denormalize:

- ▶ Proven performance bottleneck (not premature optimization!)
- ▶ Read-heavy workloads with expensive joins
- ▶ Analytics/reporting tables (data warehouses)
- ▶ Caching derived values

Cost of denormalization:

- ▶ Complex update logic (must maintain consistency)
- ▶ Increased storage
- ▶ Potential data inconsistency
- ▶ More difficult testing

Engineering practice:

- ▶ Keep OLTP (transactional) databases normalized
- ▶ Create separate denormalized views/tables for analytics
- ▶ Use materialized views when available

UML vs ER Modeling I

Semantic Differences

Two Modeling Paradigms

UML Class Diagrams model **software concepts and behavior**. ER Models describe **data semantics** at the conceptual level (entity types, identifiers, relationships).

- ▶ **Modeling Purpose**
 - ▶ UML: Describes classes, responsibilities, operations, object interactions.
 - ▶ ER: Describes persistent data semantics: entity types, identifiers, relationship types.
- ▶ **Entity Types vs Classes**
 - ▶ UML class $\neq$ ER entity type.
 - ▶ UML class may include behavior (methods); ER entity type models only data-relevant structure.
 - ▶ ER entity type represents a set of similar real-world objects (instances).

UML vs ER Modeling II

Semantic Differences

► Attributes

- UML: Attributes = properties of objects; may be derived, transient, or internal to logic.
- ER: Attributes = semantically meaningful data facts; must be persistable.
- ER allows: simple, composite, multivalued, derived attributes.

► Identification

- UML: Object identity is abstract (conceptual identity, not tied to attributes).
- ER: Identification is explicit via **entity identifiers**.
- ER distinguishes: strong vs weak entity types (weak require identifying relationship).

► Relationships / Associations

- UML: Associations model navigability and collaboration between objects in runtime.
- ER: Relationship types carry semantic meaning in the domain (not operational).

UML vs ER Modeling III

Semantic Differences

- ▶ ER relationships may have their own attributes (UML associations rarely do).
- ▶ **Multiplicity / Cardinality**
 - ▶ UML: Multiplicity constrains how many objects may be linked at runtime.
 - ▶ ER: Cardinality constraints define allowed population of entity sets (1:1, 1:N, M:N).
 - ▶ ER also uses participation constraints (total vs partial participation).
- ▶ **Generalization / Specialization**
 - ▶ UML: Inheritance = behavioral + structural reuse.
 - ▶ ER: Specialization/generalization = semantic classifications of entity types (disjoint/overlapping, total/partial).
 - ▶ No behavioral semantics in ER.

Choosing Database Model

SQL vs NoSQL

The Core Question

The Question

Every project must answer: **Which database technology fits our use case?**

This is not about "better" or "worse"

- ▶ Both SQL and NoSQL solve different problems
- ▶ The wrong choice costs time and money
- ▶ Consider: data structure, scale, consistency needs, query patterns

Common Misconception:

- ▶ "NoSQL is for scale" – SQL databases scale too (vertical and horizontal)
- ▶ "SQL is old-fashioned" – mature, reliable, well-understood
- ▶ "NoSQL is faster" – depends entirely on workload

Relational Databases (SQL)

Characteristics

Core Properties

- ▶ **Structured schema** – tables with defined columns and types
- ▶ **ACID transactions** – Atomicity, Consistency, Isolation, Durability
- ▶ **Strong consistency** – every read sees the latest write
- ▶ **Relational integrity** – foreign keys enforce relationships
- ▶ **Mature query language** – SQL is standardized and powerful

Examples: PostgreSQL, MySQL, Oracle, SQL Server, SQLite

Sweet Spot: OLTP (Online Transaction Processing) systems

- ▶ Financial systems, e-commerce, ERP, CRM
- ▶ Any system where data integrity is critical

When to Choose SQL

Choose SQL when you need:

- ▶ **Complex queries with joins** – multiple table relationships
- ▶ **Data integrity guarantees** – foreign keys, constraints, transactions
- ▶ **ACID transactions** – bank transfers, order processing
- ▶ **Well-defined schema** – data structure is known and stable
- ▶ **Aggregations and analytics** – SUM, AVG, GROUP BY, window functions
- ▶ **Mature tooling** – every language has good SQL drivers

Real-world Examples:

- ▶ E-commerce: orders, payments, inventory
- ▶ Banking: accounts, transactions, audit logs
- ▶ HR systems: employees, departments, payroll
- ▶ Booking systems: reservations, availability, customers

NoSQL Databases

Overview

Core Idea

Sacrifice some SQL features (joins, complex transactions) for:

- ▶ Horizontal scalability
- ▶ Schema flexibility
- ▶ Specialized data models
- ▶ High write throughput

Four Main Types:

- ▶ **Document** – JSON-like documents (MongoDB, CouchDB)
- ▶ **Key-Value** – simple key-value pairs (Redis, DynamoDB)
- ▶ **Column-Family** – wide columns (Cassandra, HBase)
- ▶ **Graph** – nodes and edges (Neo4j, ArangoDB)

NoSQL Types I

Document Databases

Document Databases (MongoDB, CouchDB)

Store data as **tree (forest) isomorphisms** (JSON-like documents)

Characteristics:

- ▶ Flexible schema (documents can vary)
- ▶ Nested data structures
- ▶ No joins (embed related data)
- ▶ Rich query languages

Use Cases:

- ▶ Content management systems
- ▶ User profiles and preferences
- ▶ Product catalogs with varying attributes

NoSQL Types II

Document Databases

- Real-time analytics

Example Document:

```
1 {  
2   "user_id": "123",  
3   "name": "Alice",  
4   "addresses": [  
5     {"type": "home", "city": "Warsaw"},  
6     {"type": "work", "city": "Krakow"}  
7   ]  
8 }
```


NoSQL Types

Key-Value Stores

Key-Value Stores (Redis, DynamoDB)

Simplest NoSQL model: map keys to values

Characteristics:

- ▶ Extremely fast (in-memory options)
- ▶ Simple API (GET, SET, DELETE)
- ▶ Horizontal scaling
- ▶ Values can be opaque blobs

Use cases:

- ▶ Caching (session data, page fragments)
- ▶ Rate limiting and counters
- ▶ Real-time leaderboards
- ▶ Message queues
- ▶ Distributed locks

Common pattern: Redis as cache in front of SQL database

NoSQL Types

Column-Family & Graph

Column-Family (Cassandra, HBase)

Optimized for write-heavy workloads and time-series data

Use Cases:

- ▶ Time-series data (IoT sensors, metrics)
- ▶ Event logging at massive scale
- ▶ Recommendation systems

Graph Databases (Neo4j, ArangoDB)

Optimized for relationship queries

Use Cases:

- ▶ Social networks (friends, followers)
- ▶ Recommendation engines
- ▶ Fraud detection (pattern matching)
- ▶ Knowledge graphs

When to Choose NoSQL

Choose NoSQL when you need:

- ▶ **Massive scale** – billions of records, petabytes of data
- ▶ **High write throughput** – logging, event streams, IoT
- ▶ **Schema flexibility** – frequently changing data structure
- ▶ **Simple access patterns** – key-based lookups, no complex joins
- ▶ **Specialized data models** – graphs, time-series, spatial

Decision factors:

1. Transaction requirements (ACID vs eventual consistency)
2. Data relationships (joins vs embedded documents)
3. Scale requirements (vertical vs horizontal)
4. Query complexity (SQL vs simple lookups)
5. Schema stability (fixed vs evolving)

Polyglot Persistence

Using Multiple Databases

Modern Approach

Use the right database for each part of your system

Common Pattern:

- ▶ **PostgreSQL** – core transactional data (orders, users)
- ▶ **Redis** – caching and session storage
- ▶ **Elasticsearch** – full-text search
- ▶ **MongoDB** – product catalog with varying schemas

Cost:

- ▶ Increased operational complexity
- ▶ Data synchronization challenges
- ▶ More things to monitor and maintain

So, what to do?

Start simple (one database), add others only when needed

Common mistakes in database selection

1. **Choosing NoSQL for hype** – "MongoDB is cool" is not a reason
2. **Premature optimization** – "We might need to scale" is not current need
3. **Ignoring operational costs** – NoSQL often requires more expertise
4. **Not considering queries** – your access patterns determine the right DB
5. **Assuming NoSQL = speed** – poorly designed NoSQL is slower than good SQL
6. **Forgetting about teams** – SQL expertise is more common

Golden Rule: Choose based on requirements, not trends

Application Integration

Database Integration

Architectural Layers

Clean Architecture

Separate database concerns from business logic

Typical Layering:

- ▶ **Presentation Layer** – UI, controllers, REST endpoints
- ▶ **Business Logic Layer** – services, domain logic, use cases
- ▶ **Data Access Layer** – repositories, database operations
- ▶ **Database Layer** – actual database (SQL, NoSQL)

Business logic should not depend on database
implementation details

Repository Pattern

Core Concept

Repository Pattern

Encapsulates data access logic and provides a collection-like interface

Benefits:

- ▶ **Abstraction** – business logic doesn't know about SQL/NoSQL
- ▶ **Testability** – easy to mock repositories for unit tests
- ▶ **Centralization** – all queries in one place
- ▶ **Flexibility** – can switch database implementations

Without Repository:

```
1 // Business logic directly uses SQL
2 String sql = "SELECT * FROM users WHERE id = ?";
3 User user = jdbcTemplate.queryForObject(sql, id);
```

With Repository:

```
1 // Business logic uses abstraction
2 User user = userRepository.findById(id);
```


Repository Pattern

Example Implementation

Repository Interface (in business layer):

```
1 public interface UserRepository {  
2     User findById(Long id);  
3     List<User> findAll();  
4     List<User> findByEmail(String email);  
5     void save(User user);  
6     void delete(Long id);  
7 }
```

SQL Implementation (in data access layer):

```
1 public class SqlUserRepository  
2     implements UserRepository {  
3     @Override  
4     public User findById(Long id) {  
5         // JDBC or JPA implementation  
6     }  
7     // ... other methods  
8 }
```

ORM: Object-Relational Mapping

Overview

ORM (Hibernate, Entity Framework, SQLAlchemy)

Automatically maps database tables to objects

Benefits:

- ▶ Write object-oriented code, not SQL
- ▶ Automatic CRUD operations
- ▶ Database independence (theoretically)
- ▶ Type safety
- ▶ Reduced boilerplate

Costs:

- ▶ Performance overhead (if used incorrectly)
- ▶ Complexity hidden behind abstraction
- ▶ Learning curve for advanced features
- ▶ Can generate inefficient queries

ORM Example

Entity Definition

Database Table:

```
1 CREATE TABLE users (  
2     id SERIAL PRIMARY KEY,  
3     name VARCHAR(100),  
4     email VARCHAR(100) UNIQUE  
5 );
```

Entity Class (Java + JPA):

```
1 @Entity  
2 @Table(name = "users")  
3 public class User {  
4     @Id  
5     @GeneratedValue(strategy = IDENTITY)  
6     private Long id;  
7  
8     private String name;  
9  
10    @Column(unique = true)  
11    private String email;  
12  
13    // getters, setters, constructors  
14 }
```

The N+1 Problem

Classic ORM Trap

The N+1 Problem

Most common ORM performance issue: executing N+1 queries instead of 1

Scenario: Load all users and their addresses

Bad Code (Lazy Loading):

```
1 List<User> users = userRepo.findAll(); // 1 query
2 for (User user : users) {
3     Address addr = user.getAddress(); // N queries!
4     System.out.println(addr.getCity());
5 }
```

Result: 1 query for users + N queries for addresses = N+1 queries

If you have 1000 users, you execute 1001 queries!

Solving the N+1 Problem

Solution 1: Eager Loading (JOIN FETCH)

```
1 @Query("SELECT u FROM User u JOIN FETCH u.address")  
2 List<User> findAllWithAddress();
```

Result: 1 query with JOIN

Solution 2: Batch Fetching

```
1 @BatchSize(size = 50)  
2 @OneToOne  
3 private Address address;
```

Result: 1 + ceil(N/50) queries (much better!)

Solution 3: DTO Projection

```
1 SELECT new UserAddressDTO(u.name, a.city)  
2 FROM User u JOIN u.address a
```

Result: 1 query, only needed data

When to Use ORM vs Raw SQL

Use ORM for:

- ▶ Simple CRUD operations
- ▶ Standard queries with predicates
- ▶ When objects map cleanly to tables
- ▶ Rapid prototyping
- ▶ When team lacks SQL expertise

Use Raw SQL for:

- ▶ Complex analytical queries
- ▶ Bulk operations (updates, deletes)
- ▶ Performance-critical paths
- ▶ Database-specific features (window functions, CTEs)
- ▶ Reporting and aggregations

Schema Migrations

Evolving Your Database

The Problem

Database schema must evolve with application code

Why Migrations Matter:

- ▶ Database changes need to be versioned like code
- ▶ Deployments must apply schema changes reliably
- ▶ Teams need consistent database state
- ▶ Rollbacks must be possible

Migration Tools:

- ▶ **Flyway** (Java) – SQL-based migrations
- ▶ **Liquibase** (Java) – XML/YAML/SQL migrations
- ▶ **Alembic** (Python) – SQLAlchemy migrations
- ▶ **Entity Framework Migrations** (.NET)

Migration Example

Flyway

Migration File: V1__create_users_table.sql

```
1 CREATE TABLE users (  
2     id SERIAL PRIMARY KEY,  
3     name VARCHAR(100) NOT NULL,  
4     email VARCHAR(100) UNIQUE NOT NULL  
5 );
```

Migration File: V2__add_phone_to_users.sql

```
1 ALTER TABLE users  
2 ADD COLUMN phone VARCHAR(20);
```

Principles for migrations:

- ▶ Migrations are **sequential and immutable**
- ▶ Each migration has a **version number**
- ▶ Never modify existing migrations
- ▶ Always test migrations on staging first
- ▶ Write both up and down migrations when possible

Migration Best Practices

1. **Version control migrations** – treat like code
2. **Make migrations idempotent when possible** – can run multiple times safely
3. **Test on production-like data** – edge cases matter
4. **Plan for zero-downtime deployments:**
 - ▶ Add columns as nullable first
 - ▶ Backfill data in separate step
 - ▶ Make NOT NULL in final migration
5. **Keep migrations small and focused** – easier to debug
6. **Document breaking changes** – alert team members
7. **Have rollback plan** – not all changes are reversible

Design Principles & Anti-patterns

Database Design Principles

Software Engineering Perspective

- 1. Start from the domain model**
 - ▶ Map business entities first, optimize later
 - ▶ Preserve business rules in database constraints
- 2. Design for evolvability**
 - ▶ Assume requirements will change
 - ▶ Use migrations from day one
 - ▶ Keep schema flexible where needed
- 3. Avoid premature optimization**
 - ▶ Start normalized
 - ▶ Measure before optimizing
 - ▶ Profile queries to find actual bottlenecks
- 4. Automate everything**
 - ▶ Migrations in CI/CD pipeline
 - ▶ Automated backups
 - ▶ Monitoring and alerting

Common Anti-patterns

What to Avoid

1. **God tables** – one huge table with 100+ columns
 - ▶ Hard to query efficiently
 - ▶ Difficult to maintain
 - ▶ Usually indicates missing normalization
2. **Entity-Attribute-Value (EAV)** – generic "flexible" schema
 - ▶ Looks flexible, actually makes queries impossible
 - ▶ No type safety, no constraints
 - ▶ Use JSON columns or NoSQL instead
3. **Storing logs in transactional database**
 - ▶ Logs grow unbounded
 - ▶ Slows down OLTP operations
 - ▶ Use dedicated log storage (ELK, Splunk)

Common Anti-patterns

More Pitfalls

4. Using ORM for everything

- ▶ Complex queries become unmaintainable
- ▶ N+1 problems everywhere
- ▶ Use raw SQL when appropriate

5. No indexing strategy

- ▶ "We'll add indexes if it's slow"
- ▶ Every query does full table scan
- ▶ Indexes should be part of initial design

6. Missing foreign keys

- ▶ "We'll enforce it in application code"
- ▶ Data integrity bugs creep in
- ▶ Orphaned records accumulate

Common Anti-patterns

Architectural Issues

7. Shared database in microservices

- ▶ Tight coupling between services
- ▶ Cannot deploy independently
- ▶ Breaks microservice principles

8. No clear data ownership

- ▶ Multiple services write to same tables
- ▶ Race conditions and conflicts
- ▶ Unclear responsibility

9. Storing computed values redundantly

- ▶ Without proper update logic
- ▶ Values get out of sync
- ▶ Use views or compute on read